

# Agentic Revenue, Pricing & Promotion Intelligence

Engineering Handbook

*A CPG / Retail platform where Claude and LangGraph orchestrate reasoning  
and deterministic models produce every number.*

Stage 1, Steps 1-5 complete  
Generated 2026-08-27 | commit 95043ed  
534 tests passing | ruff, mypy, bandit clean

## Contents

|                                                        |    |
|--------------------------------------------------------|----|
| Contents .....                                         | 2  |
| How to read this handbook .....                        | 4  |
| Part I - Orientation .....                             | 5  |
| 1. What this platform is .....                         | 5  |
| 2. The two-stage architecture .....                    | 5  |
| 3. Layered architecture.....                           | 6  |
| 4. Technology stack .....                              | 6  |
| 5. Repository map.....                                 | 7  |
| Part II - Foundations .....                            | 8  |
| Step 1 - Skeleton and the three seams .....            | 8  |
| Seam 1 - DataRepository .....                          | 8  |
| Seam 2 - the ToolResult envelope .....                 | 8  |
| Seam 3 - the DI container .....                        | 8  |
| Defects found in Step 1 .....                          | 9  |
| Step 2 - Synthetic data with hidden ground truth ..... | 10 |
| Scale and shape .....                                  | 10 |
| The six deliberate confounders .....                   | 10 |
| Hidden ground truth.....                               | 11 |
| Defects found in Step 2 .....                          | 11 |
| Step 3 - Data access, contracts and features .....     | 12 |
| Availability classes .....                             | 12 |
| Point-in-time correctness as a type constraint.....    | 12 |
| The feature pipeline .....                             | 13 |
| Defects found in Step 3 .....                          | 13 |
| Part III - The models .....                            | 14 |
| The distinction that carries both steps.....           | 14 |
| Step 4 - Baseline sales model.....                     | 15 |
| The two design decisions.....                          | 15 |
| Results .....                                          | 15 |
| Step 5 - Demand forecasting model .....                | 17 |
| The horizon dataset.....                               | 17 |
| Why one model rather than four .....                   | 17 |
| The embargo.....                                       | 18 |

|                                                    |    |
|----------------------------------------------------|----|
| Prediction intervals.....                          | 18 |
| The training pipeline.....                         | 18 |
| Results .....                                      | 19 |
| Behaviour beyond the end of the data.....          | 19 |
| Part IV - Cross-cutting concerns.....              | 21 |
| Leakage: defence in depth .....                    | 21 |
| Testing philosophy.....                            | 22 |
| Reproducibility.....                               | 22 |
| Serving and the agent contract .....               | 23 |
| Part V - Operations .....                          | 24 |
| Setup.....                                         | 24 |
| Generating data.....                               | 24 |
| Training the models.....                           | 24 |
| Quality gates.....                                 | 24 |
| Configuration reference .....                      | 24 |
| Databricks migration .....                         | 25 |
| Part VI - Retrospective .....                      | 26 |
| The defect catalogue.....                          | 26 |
| Defects that produced wrong numbers silently ..... | 26 |
| Defects in the diagnostics themselves .....        | 26 |
| Process defects .....                              | 27 |
| Known limitations.....                             | 27 |
| Roadmap.....                                       | 28 |
| Talking points.....                                | 28 |

## How to read this handbook

This document is the single reference for the platform: what it does, why each decision was made, what went wrong along the way, and how to run and extend it. It is organised so you can enter at any level.

| If you want to...                                 | Read                           |
|---------------------------------------------------|--------------------------------|
| Understand the shape of the system in ten minutes | Part I - Orientation           |
| Know why the data is synthetic and what that buys | Part II, Step 2                |
| Understand how leakage is prevented structurally  | Part II Step 3, and Part IV    |
| Understand the models and their measured results  | Part III - Steps 4 and 5       |
| Run the platform or retrain a model               | Part V - Operations            |
| Learn from the mistakes rather than repeat them   | Part VI - the defect catalogue |
| Prepare to discuss this in an interview           | Part VI - talking points       |

**A note on tone.** Numbers in this document are measured, not estimated. Where something is uncertain, underspecified, or was got wrong and later corrected, it says so. The defect catalogue in Part VI is deliberately unflattering - it is the most useful section in the document.

## Part I - Orientation

### 1. What this platform is

A commercial analytics platform for consumer packaged goods and retail. It answers questions a category manager actually asks - how much will we sell, did that promotion work, what should this product cost - and it answers them with a specific division of labour:

- **Claude and LangGraph orchestrate reasoning.** They decide which question is being asked, which analyses to run, in what order, and how to explain the result.
- **Deterministic models produce every number.** No figure in any answer originates from a language model. Every quantity is traceable to a fitted model, a dataset version and a code commit.

**The governing constraint.** The LLM must never invent a numerical result. This is not a stylistic preference - it is what makes the platform auditable. An agent that can estimate a number is an agent whose output cannot be checked.

### 2. The two-stage architecture

The platform is built twice over: once locally, where iteration is fast and everything is inspectable, and once on Databricks, where it would run in production. The second build is a redeployment rather than a rewrite, and that claim is enforced by three seams rather than asserted in a document.

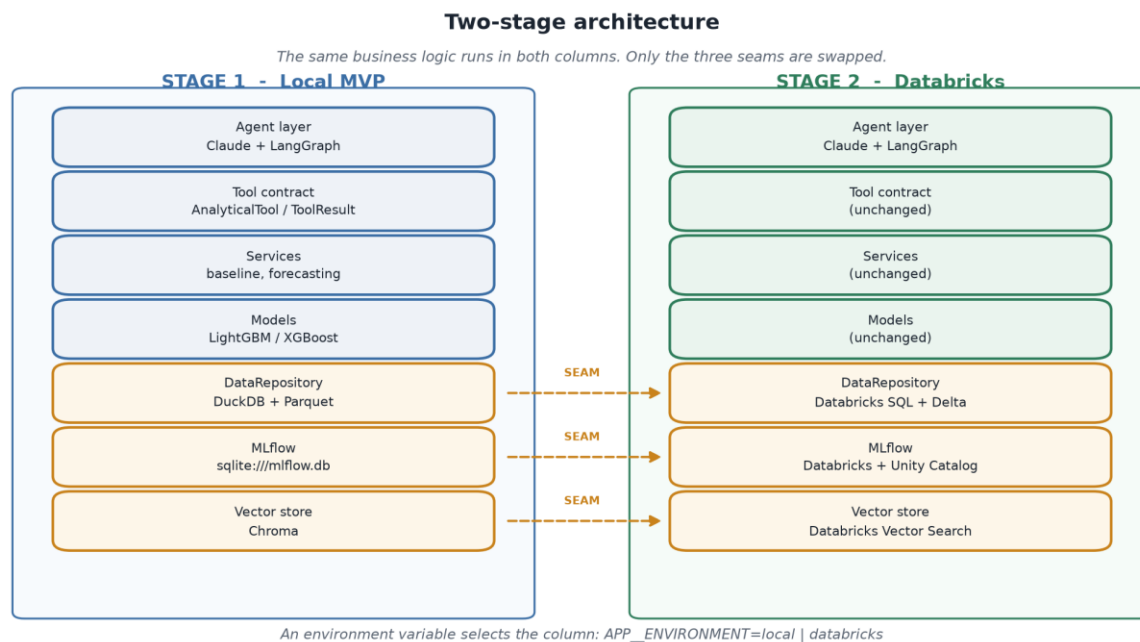


Figure 1 - Only three components differ between local and production.

Everything above the seams - services, models, feature logic, the tool contract, the agent layer - is identical in both columns. A model trained locally moves to Stage 2 without a code change, because it never learns where its data came from.

**No AWS.** This platform is Databricks-native in production. AWS is not used anywhere - no S3, no ECR, no Lambda, no managed database. The constraint is deliberate and is reflected in the migration design in Part V.

### 3. Layered architecture

Within either column, dependencies point strictly downward. A model may use the feature layer; the feature layer may not know a model exists.

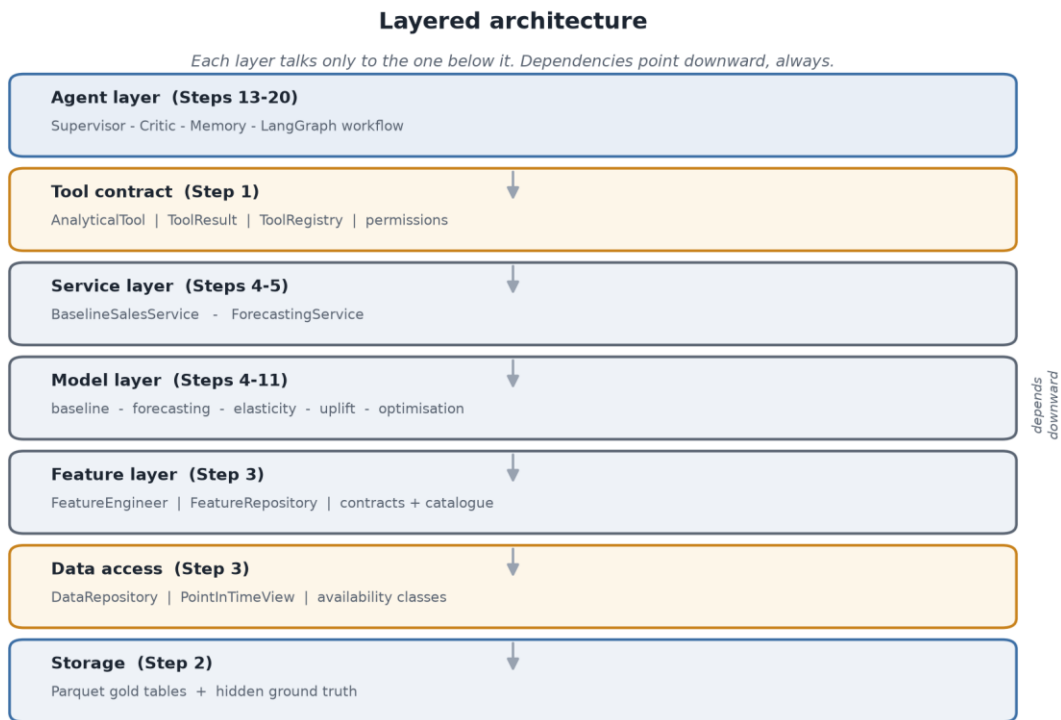


Figure 2 - Seven layers. The two amber layers are where implementations swap.

### 4. Technology stack

| Concern             | Stage 1 (local)                              | Stage 2 (production)      |
|---------------------|----------------------------------------------|---------------------------|
| Language            | Python 3.12+, managed by uv                  | same                      |
| Analytics store     | DuckDB over Parquet                          | Databricks SQL over Delta |
| Application state   | SQLite                                       | Databricks SQL            |
| Feature engineering | pandas                                       | PySpark                   |
| Models              | LightGBM, XGBoost, scikit-learn, statsmodels | same                      |
| Optimisation        | OR-Tools                                     | same                      |
| Experiment tracking | MLflow on SQLite                             | Databricks MLflow         |
| Model registry      | local MLflow registry                        | Unity Catalog             |
| Vector store        | Chroma                                       | Databricks Vector Search  |
| Reasoning           | Claude (Anthropic API)                       | same                      |
| Orchestration       | LangGraph                                    | same                      |
| Contracts           | Pydantic, Pandera                            | same                      |

| API           | FastAPI                    | Databricks Model Serving |
|---------------|----------------------------|--------------------------|
| Quality gates | ruff, mypy, bandit, pytest | same                     |

## 5. Repository map

```

app/          application layer
  config/     Settings - one class per section, env-prefixed
  services/   Container (DI), baseline_service, forecast_service
  tools/      AnalyticalTool ABC, ToolResult envelope, registry
  schemas/    Pydantic contracts: domain, tool_contract, baseline, forecast
  observability/ structlog logging, trace context, metrics
  api/        FastAPI app and routes

data/         data layer
  generation/ the synthetic data simulator (Step 2)
  repositories/ DataRepository ABC, Local + Databricks, PointInTimeView
  contracts/   Pandera table schemas

features/     feature layer (Step 3)
  contracts/   catalogue, specs, config loader, feature versioning
  engineering/ panel helpers, demand, pricing, promotion, inventory, temporal
  datasets/    per-model dataset builders
  repositories/ FeatureRepository ABC + local implementation

ml/          model layer
  base.py      AnalyticalModel ABC, ModelMetadata, error types
  baseline/    Step 4 - baseline sales
  forecasting/ Step 5 - demand forecasting
  <others>/    interface-only stubs for Steps 6-11

tests/        534 tests, marked by concern
configs/      YAML: dataset profiles, feature selection, model config
scripts/      generate_data, train_baseline, train_forecast
docs/         this handbook, model cards, migration designs
notebooks/    exploration and validation, one folder per step

```

## Part II - Foundations

### Step 1 - Skeleton and the three seams

Step 1 built no features and trained no models. It established the boundaries that make the remaining nineteen steps possible, and the reason it comes first is that these boundaries are almost impossible to retrofit.

#### Seam 1 - DataRepository

An abstract base class defining every read the platform performs. Business logic receives a repository and never asks which implementation it got.

```
class DataRepository(ABC):
    @abstractmethod
    def get_sales(self, *, product_ids=None, store_ids=None,
                  start_date=None, end_date=None, as_of_date=None,
                  max_rows=None) -> pd.DataFrame: ...

    def as_of(self, as_of_date: date) -> PointInTimeView:
        return PointInTimeView(self, as_of_date)
```

#### Seam 2 - the ToolResult envelope

Every analytical capability returns the same shape. `AnalyticalTool.run()` is marked `@final`, so a subclass cannot bypass validation, timing, tracing or error wrapping - it implements only `_execute`.

| Field                                         | Purpose                                                     |
|-----------------------------------------------|-------------------------------------------------------------|
| status                                        | success / partial / error / invalid_input / timeout         |
| model_name, model_version,<br>dataset_version | Provenance - which artifact produced this number            |
| result                                        | The payload, as plain JSON                                  |
| confidence                                    | Measured, or absent. Never invented                         |
| assumptions                                   | What the caller must know to use the number responsibly     |
| warnings                                      | Caveats; a non-empty list downgrades status to partial      |
| error                                         | Code, message, and a recoverable flag for agent re-planning |
| trace_id                                      | Ties the result to its log lines                            |

#### Seam 3 - the DI container

One class with cached properties, keyed on `APP__ENVIRONMENT`. Components are built lazily, so constructing the container never requires a trained model or an API key - a missing artifact surfaces at the point of use, not at boot.

**Design decision.** Not a DI framework. dependency-injector and friends buy scoping, auto-wiring and declarative overrides; with four factories and one switch they would add a dependency and a mental model in exchange for nothing.



### Defects found in Step 1

| Symptom                                          | Cause                                                          | Fix                                          |
|--------------------------------------------------|----------------------------------------------------------------|----------------------------------------------|
| Settings failed to construct                     | **_SECTION_CONFIG collided with an explicit env_prefix keyword | Standalone dict constant, spread per section |
| 46 test errors, AttributeError on every log call | structlog.stdlib.add_logger_name with PrintLoggerFactory       | Switched to structlog.stdlib.LoggerFactory() |
| bandit B101                                      | assert used for type narrowing - stripped under python -O      | Explicit local-variable narrowing            |

## Step 2 - Synthetic data with hidden ground truth

The dataset is generated from a structural causal model rather than sampled from a distribution. That distinction is what makes every later step testable: because the true elasticities, promotion lifts and latent demand are known, a model can be checked against the answer rather than against its own fit.

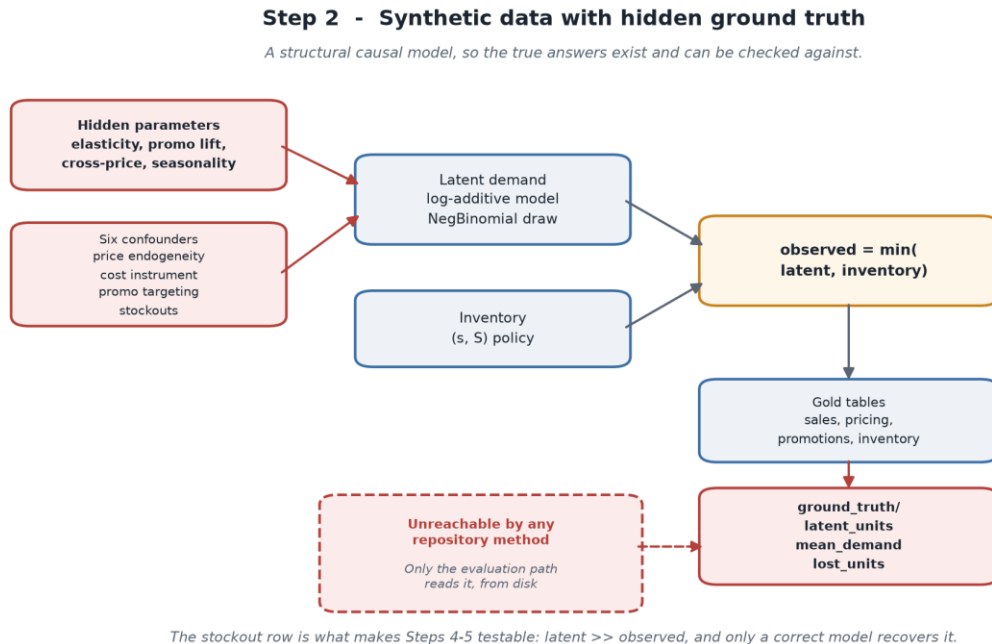


Figure 3 - Demand is simulated, then censored by inventory. Both are retained.

### Scale and shape

| Property                         | Value                                                   |
|----------------------------------|---------------------------------------------------------|
| Period                           | 2023-01-01 to 2025-12-31 (1,096 days)                   |
| Products / stores                | 300 / 200                                               |
| <b>Real product-store series</b> | <b>6,128</b> - each product is listed in ~10% of stores |
| Daily sales rows                 | 6,716,288                                               |
| Total rows across all tables     | 23.6 million                                            |
| Gold tables                      | 11                                                      |

### The six deliberate confounders

A dataset where price moves at random would make elasticity trivial to recover and the platform's causal machinery pointless. These confounders are what give Steps 6-8 something real to solve.

| Confounder                                    | What it creates                                    |
|-----------------------------------------------|----------------------------------------------------|
| Price endogeneity (0.45)                      | Price responds to demand, biasing naive elasticity |
| Commodity-cost instrument (0.55 pass-through) | A <b>valid IV</b> - moves price, not demand        |

|                                         |                                                             |
|-----------------------------------------|-------------------------------------------------------------|
| Randomised price tests (18% of periods) | A clean experimental subset to validate against             |
| Promotion targeting (0.40)              | Promotions scheduled toward seasonal peaks - selection bias |
| Competitor-cost correlation (0.40)      | Competitor prices move with shared costs                    |
| Endogenous stockouts                    | Stockouts happen <b>because</b> demand spiked               |

**The finding that matters most.** Latent demand during a stockout runs 1.57x normal. Stockouts are caused by demand, not merely correlated with it. This single fact invalidated the original stockout diagnostic in Step 4 - see Part III.

### Hidden ground truth

data/local/ground\_truth/ holds the true parameters and the uncensored demand. It is **structurally unreachable** through any repository method: the local repository registers only the gold directory, so no get\_\* call and no SQL query can reach it. The single sanctioned reader is the evaluation path, which reads from disk directly.

### Defects found in Step 2

| Symptom                                         | Cause                                                                                                                   | Why it mattered                                                            |
|-------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| Stockouts were impossible - 170 days of cover   | Replenishment ordered against <b>on-hand stock, not inventory position</b> , stacking duplicate orders during lead time | The censoring mechanism that Steps 4-5 are validated against did not exist |
| Data changed between identical runs             | Chunk RNG used <code>SeedSequence.spawn()</code> , which mutates the parent's child counter                             | Reproducibility - the headline claim of the step - was false               |
| Elasticity unidentifiable at dev scale          | Validation sampled the <b>first</b> partitions, giving a 3-month window                                                 | A correct simulator looked broken                                          |
| Transaction table 19x too large                 | Sampling rate mis-set                                                                                                   | Generation time and disk                                                   |
| <code>__init__.py</code> files silently emptied | <code>New-Item -ItemType File -Force</code> truncates existing files                                                    | Packages stopped importing                                                 |

## Step 3 - Data access, contracts and features

Step 3 is where leakage prevention became structural rather than procedural. The central insight is that the naive rule - clamp every table to the as-of date - is wrong, and being wrong in the safe direction still destroys information the platform needs.

### Availability classes

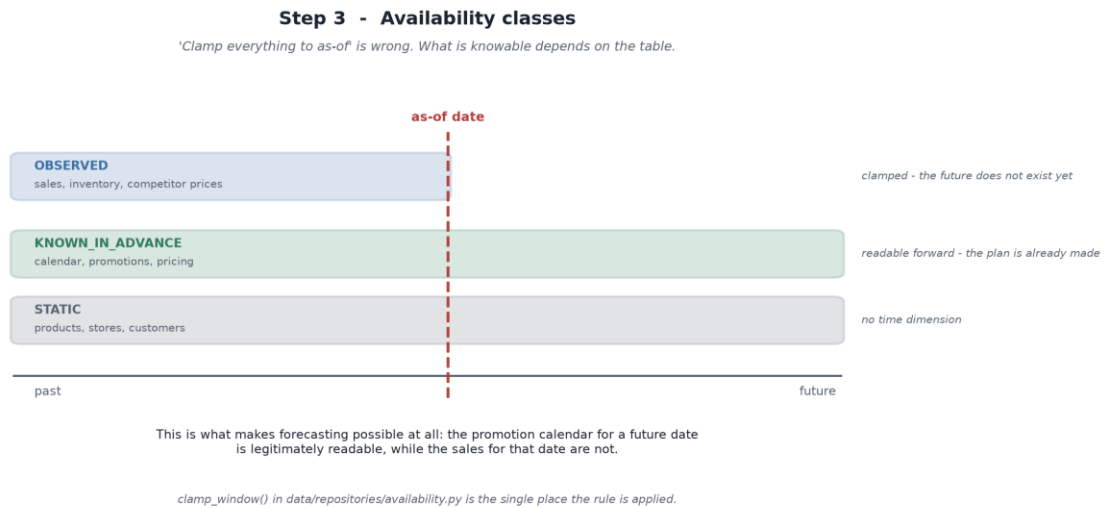


Figure 4 - What is knowable depends on the table, not on a global rule.

Sales for a future date do not exist. The promotion calendar for that same date does - it was planned weeks ago. Blanket clamping would delete exactly the information a forecast depends on, and Step 5 would have been impossible.

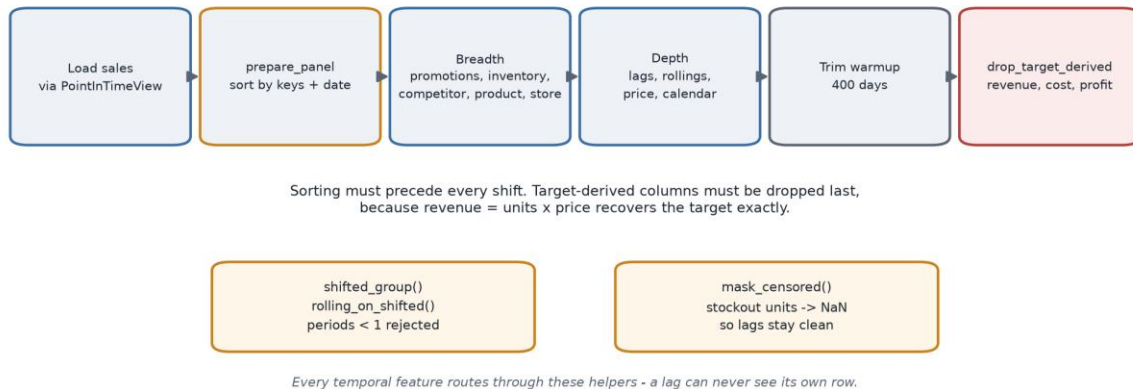
### Point-in-time correctness as a type constraint

FeatureEngineer requires a PointInTimeView and raises TypeError on a bare repository. Leakage through the feature layer is therefore not a discipline question - it is a construction that will not run.

## The feature pipeline

### Step 3 - Feature engineering order

*The sequence is load-bearing: breadth before depth, target-derived dropped last.*



*Figure 5 - The order is load-bearing, not incidental.*

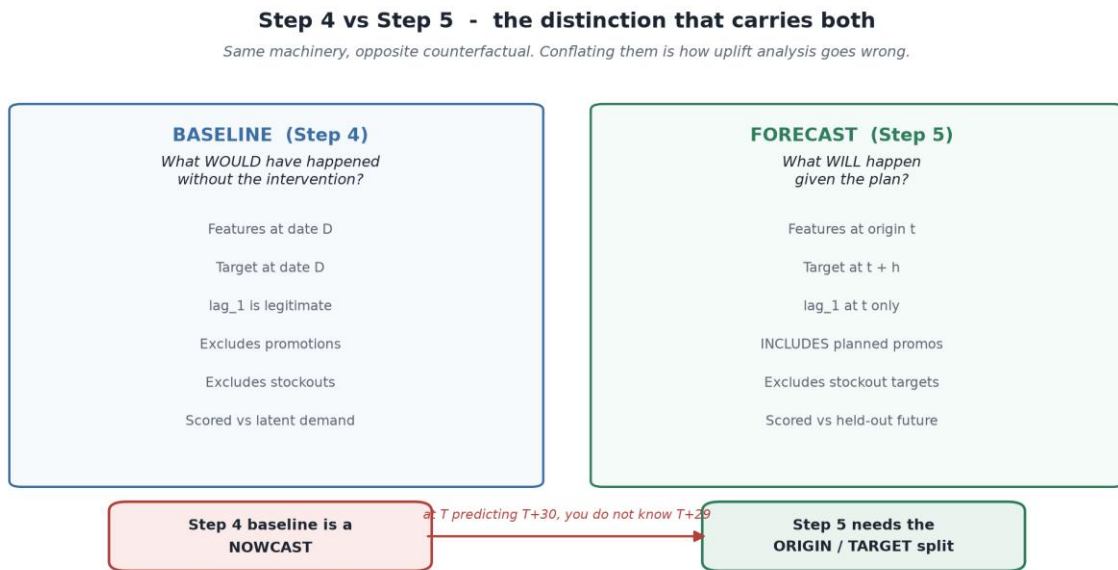
- **Sorting precedes every shift.** A lag computed on an unsorted panel is silently wrong.
- **Target-derived columns are dropped last.** revenue = units x price recovers the target exactly, and a model given revenue learns arithmetic rather than demand.
- **Every temporal feature routes through shifted\_group or rolling\_on\_shifted.** A window can never include its own row; periods < 1 is rejected outright.

### Defects found in Step 3

| Symptom                                                      | Cause                                                                      | Fix                                                                    |
|--------------------------------------------------------------|----------------------------------------------------------------------------|------------------------------------------------------------------------|
| A query returned exactly 100,000 of 6,716,288 rows, silently | Row cap applied with no signal to the caller                               | ResultTruncatedError - a truncated analysis is worse than a failed one |
| Near-perfect model accuracy                                  | revenue, cost, gross_profit reached the feature matrix                     | drop_target_derived() inside the engineer                              |
| Panels came back inexplicably sparse                         | Products and stores sampled independently, producing never-co-listed pairs | Co-listed sampling path                                                |
| channel parameter vanished from a method                     | Regex-based patching of Python source                                      | Use the editor, never regex, on code                                   |

## Part III - The models

### The distinction that carries both steps



*Figure 6 - Same machinery, opposite counterfactual.*

A baseline estimates what **would** have happened without an intervention. A forecast predicts what **will** happen including planned interventions. Conflating them is the most common way uplift analysis goes wrong: a forecast that includes the promotion measures the promotion against itself and reports roughly zero uplift.

## Step 4 - Baseline sales model

The question: what would sales have been under normal conditions - no promotion running, stock available? Promotion uplift is measured against it, root cause compares actual to it, and scenario simulation starts from it. A biased baseline does not fail loudly; it propagates a consistent distortion into every downstream number.

### The two design decisions

**Promotional contamination.** Two defensible approaches, both with a real bias:

| Approach                                                            | Strength                             | Bias                                                                                                         |
|---------------------------------------------------------------------|--------------------------------------|--------------------------------------------------------------------------------------------------------------|
| <b>C</b> - train on non-promotional rows only                       | Clean counterfactual semantics       | Promotions target seasonal peaks, so dropping them under-represents high season and <b>overstates uplift</b> |
| <b>B</b> - promotion features as controls, predict with them zeroed | Uses all rows, avoids selection bias | Asks a tree model to extrapolate to a feature combination it may rarely have seen                            |

Neither argument settles it, so both were built and scored against true demand. Approach C won by 0.2 points - too narrow to generalise, and reported as such.

**Stockout censoring.** Observed sales during a stockout measure availability, not demand. Stockout rows are excluded from training, and censored values are not lagged forward.

**The finding that reshaped the step.** The first full run disqualified both LightGBM candidates. The feature importances explained why: `closing_inventory_lag_1` was the single most important feature. Given inventory, the model learns that low stock predicts low sales - which is true, and is exactly the censoring relationship the step exists to avoid. Excluding stockout rows does not prevent it; the relationship is learned from partially-depleted rows and extrapolated to zero stock.

| LightGBM configuration     | Stockout lift | Recovers (correct ~0.64) | Outcome             |
|----------------------------|---------------|--------------------------|---------------------|
| Inventory features present | 1.12          | 0.30                     | <b>disqualified</b> |
| Supply features excluded   | <b>2.48</b>   | <b>0.68</b>              | selected            |

*Accuracy moved only 40.4% to 40.2% WMAPE. That is the telling part: those columns were contributing censoring signal rather than demand signal, so removing them cost nothing and fixed the diagnostic outright.*

### Results

| Metric                         | Value                 | Reading                                        |
|--------------------------------|-----------------------|------------------------------------------------|
| Selected model                 | lightgbm__exclude     | Beat the seasonal naive by 13.9 points         |
| Latent WMAPE                   | 40.4%                 | Against true demand, not observed sales        |
| <b>Irreducible noise floor</b> | <b>35.0%</b>          | What a model knowing the true mean would score |
| <b>Ratio to floor</b>          | <b>1.15x</b>          | Most remaining error is unlearnable noise      |
| Stockout lift                  | 2.48x                 | Sees through the censoring                     |
| Interval coverage              | 92.0% on 587,603 rows | Nominal 90% - calibrated                       |

|          |                |                                    |
|----------|----------------|------------------------------------|
| Backtest | 39.0% +/- 0.7% | Stable across four expanding folds |
|----------|----------------|------------------------------------|

**An unresolved tension, stated rather than resolved.** The design argues bias matters more than error for a baseline, but the selection rule ranks on WMAPE - and here they disagree. LightGBM is 3.3 points more accurate; Ridge is nearly unbiased (-0.5% vs +6.7%). The over-prediction understates uplift rather than inventing it, which is the safer direction, but adding a bias term to the criterion is a real decision and belongs with Step 6.



## Step 5 - Demand forecasting model

Step 4's baseline is a nowcast - it predicts units at date D using features at D, including yesterday's sales. That is valid for a historical counterfactual and invalid for forecasting: standing at as-of T predicting T+30, you do not know T+29's sales. Every design decision in Step 5 follows from that gap.

### The horizon dataset

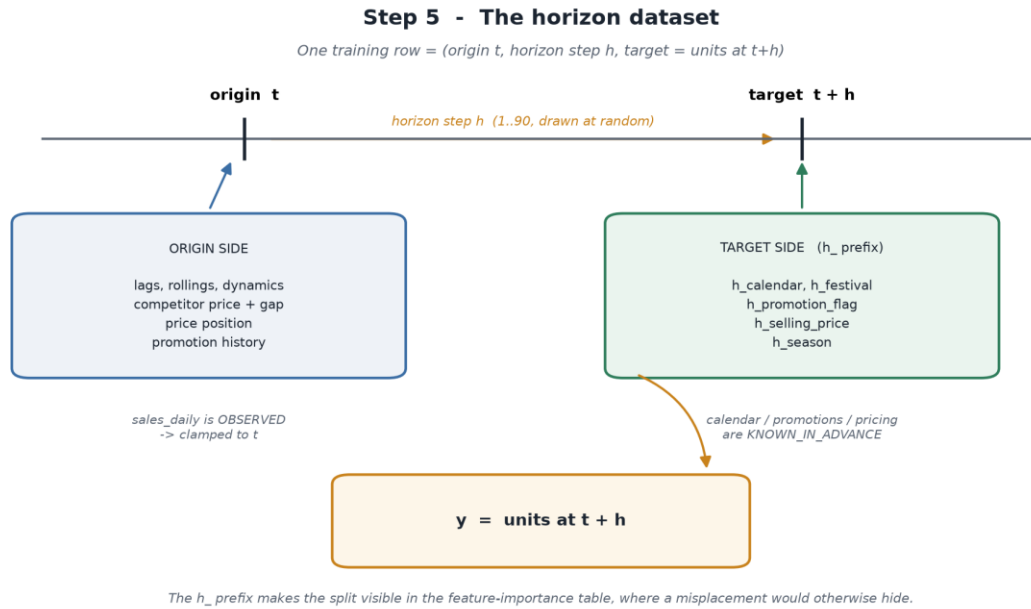


Figure 7 - Each feature is placed by asking whether it is knowable at the origin.

| Feature family             | Sourced at | Why legitimate                    |
|----------------------------|------------|-----------------------------------|
| lags, rollings, dynamics   | origin t   | sales_daily is OBSERVED           |
| competitor price and gap   | origin t   | OBSERVED - not knowable forward   |
| calendar, festival, season | target t+h | calendar is KNOWN_IN_ADVANCE      |
| planned promotion          | target t+h | promotions is KNOWN_IN_ADVANCE    |
| planned price              | target t+h | pricing is KNOWN_IN_ADVANCE       |
| horizon_step               | -          | Lets one model span every horizon |

Target-side columns carry an h\_ prefix, so the origin/target split is visible in the feature-importance table rather than being a fact you must remember.

### Why one model rather than four

- **Four per-horizon models on a cumulative target** - dispositive: cannot produce the daily path the interface requires, and the nested totals would be mutually incoherent.
- **Recursive one-step-ahead** - the killer is feature-distribution collapse, not error compounding. Feeding conditional-mean predictions forward drives rolling standard deviations and volatility toward zero, so by h=30 the model sees inputs it never saw.

- **Four direct models on daily targets** - 4x the compute, less data per model, no benefit once  $h$  is already a feature.

## The embargo

### Step 5 - Temporal split with an embargo

*Without the gap, a boundary training origin has its target inside the evaluation fold.*

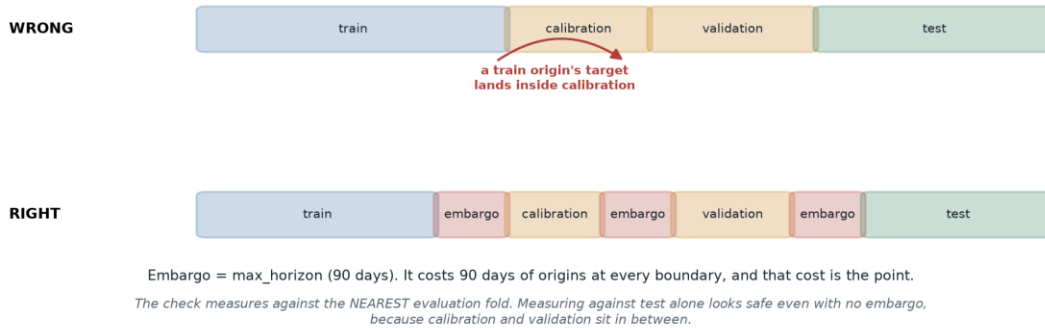


Figure 8 - Without the gap, a boundary training origin is scored on its own outcome.

## Prediction intervals

Two things Step 4's single scalar quantile could not do. **Width must grow with horizon**, so calibration is per horizon bucket (Mondrian conformal). And **the horizon total needs its own calibration** - summing daily bounds assumes the daily errors move together perfectly, and if they were independent the sum would be too wide by roughly the square root of 90.

## The training pipeline

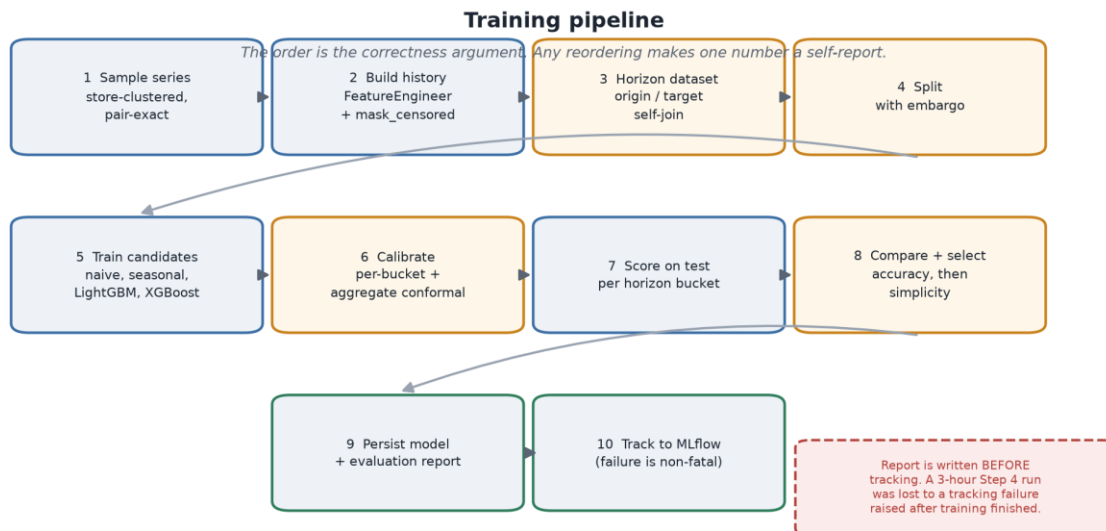


Figure 9 - The order is the correctness argument.

## Results

| Model                  | WMAPE | Bias   | Mean FVA | Train |
|------------------------|-------|--------|----------|-------|
| xgboost                | 43.8% | +8.4%  | +12.6 pp | 34.4s |
| lightgbm               | 47.5% | +19.9% | +9.5 pp  | 11.3s |
| horizon_seasonal_naive | 56.0% | +7.2%  | -        | 0.4s  |
| horizon_naive          | 78.1% | +36.1% | -19.8 pp | 0.4s  |

800 series, 548,754 rows, 475 seconds. Forecast Value Added is **positive at every horizon bucket** (+11 to +15 points), so there is no horizon at which the naive benchmark would be the better choice.

| Aggregation level | Series | WMAPE |
|-------------------|--------|-------|
| product x store   | 45,295 | 43.6% |
| product           | 24,752 | 35.7% |
| store             | 4,857  | 18.7% |
| category          | 854    | 15.0% |
| region            | 610    | 11.8% |
| total             | 122    | 9.6%  |

Error falls by a factor of 4.5 from SKU to total. Bottom-up aggregation is exactly coherent by construction, so nothing is reconciled - this table quantifies the price of that choice, and answers 'should I trust the regional number more than the SKU number?' with a magnitude.

**Reported rather than smoothed.** The horizon gradient is shallow - 43.6% at h1-3 against 44.7% at h57-90, only 1.1 points over three months. At 1.25x the noise floor there are only ~9 points of learnable signal in total, so a steep curve is arithmetically unavailable. That makes the gradient weak evidence about join correctness on this dataset, and the leakage tests carry that argument instead.

**The parameter trap worth remembering.** XGBoost first scored 82.9% WMAPE at +58% bias and looked far worse than LightGBM. The cause was semantics: `min_child_weight` sums Hessians, and under `count:poisson` the Hessian is approximately  $\mu$ , so the parameter scales with the target level. LightGBM's `min_child_samples` counts rows. Setting both to 50 gave XGBoost ~38x less regularisation. Scaled to  $50 \times \text{mean}(y)$  it scores 43.8%. Reporting the unscaled run would have put a confident and completely false line in the comparison table.

## Behaviour beyond the end of the data

The calendar, promotion schedule and price plan all end 2025-12-31, so a 90-day forecast is only fully informed from as-of 2025-10-02. Requests past that are **refused** with a recoverable error naming the latest workable as-of.

```
insufficient_data: a 90-day horizon from 2025-12-01 reaches 2026-03-01, but the
calendar, promotion schedule and price plan end 2025-12-31. Forecasting past that
would mean assuming no promotions are planned, which biases those days low. The
latest as-of that supports a 90-day horizon is 2025-10-02.
```

*The alternative - assume no promotion runs and carry the last price forward - produces a number that is systematically low and indistinguishable from a real forecast.*



## Part IV - Cross-cutting concerns

### Leakage: defence in depth

Leakage has the worst signal-to-noise of any failure mode in applied ML: it never raises, never warns, and makes every number look better. The only defence is to assert the absence of something, repeatedly, from several angles.



Figure 10 - Five independent layers, each catching what the others miss.

**The test that makes the rest meaningful.** A test that has never failed proves nothing about its ability to detect anything. So the suite plants the exact bug the design prevents - horizon\_features\_from\_target=True sources origin features from the target row - and asserts the detector fires. Without that, the mutation test is unfalsifiable.

| Test                      | Asserts                                                                             |
|---------------------------|-------------------------------------------------------------------------------------|
| Mutation (T1)             | Corrupting all OBSERVED data after a cutoff leaves training features byte-identical |
| Falsifiability (T2)       | <b>T1 fails</b> when the bug is deliberately planted                                |
| Arithmetic (T3)           | lag_7 at origin t equals units at t-7, reconstructed by hand from the source panel  |
| Horizon monotonicity (T4) | Error over long horizons exceeds short ones                                         |
| Noise floor (T5)          | Nothing scores below the irreducible 35% WMAPE                                      |
| Train/serve (T6)          | Both feature paths produce identical vectors                                        |
| Embargo (T7)              | No training target lands inside an evaluation fold                                  |

*T6 has already earned its place twice: it caught festival columns going missing over short serving windows, and categorical dtypes being inferred per frame.*

## Testing philosophy

- **Reconstruct expectations by hand.** A test that calls the implementation to compute its own expected value cannot detect a bug in that implementation.
- **Prefer behavioural assertions.** 'Error grows with horizon' survives a refactor that renames every feature; a test naming columns does not.
- **Contextualise, do not hard-code.** Comparing WMAPE against a measured noise floor is a statement about the model. Comparing it against a threshold that happened to pass is a statement about how the fixture is tuned.
- **Test the failure path first.** Most of what matters about a service is how it behaves when something is missing, and that is the path least likely to be exercised by hand.

| Marker      | Covers                                                        |
|-------------|---------------------------------------------------------------|
| unit        | Fast, isolated, no I/O                                        |
| data        | Synthetic generation and dataset invariants                   |
| features    | Feature engineering and the feature repository                |
| models      | ML model behaviour                                            |
| leakage     | Point-in-time correctness - the property Steps 4-11 depend on |
| statistical | Relationship recovery against known ground truth              |
| integration | API and on-disk artifacts                                     |

## Reproducibility

Every training run records the seed, dataset version, feature version, code commit, split boundaries, hyper-parameters, and a **config fingerprint** - one hash over the entire configuration. Two runs sharing a fingerprint used the same setup; two that differ are not comparable, and the difference is discoverable rather than argued about.

**A guard learned expensively.** The evaluation report is written **before** MLflow tracking, and a tracking failure never fails the run. A three-hour Step 4 run was lost when MLflow rejected its own default store after every model had already been fitted. Bookkeeping must never destroy the thing it is keeping books on.

## Serving and the agent contract

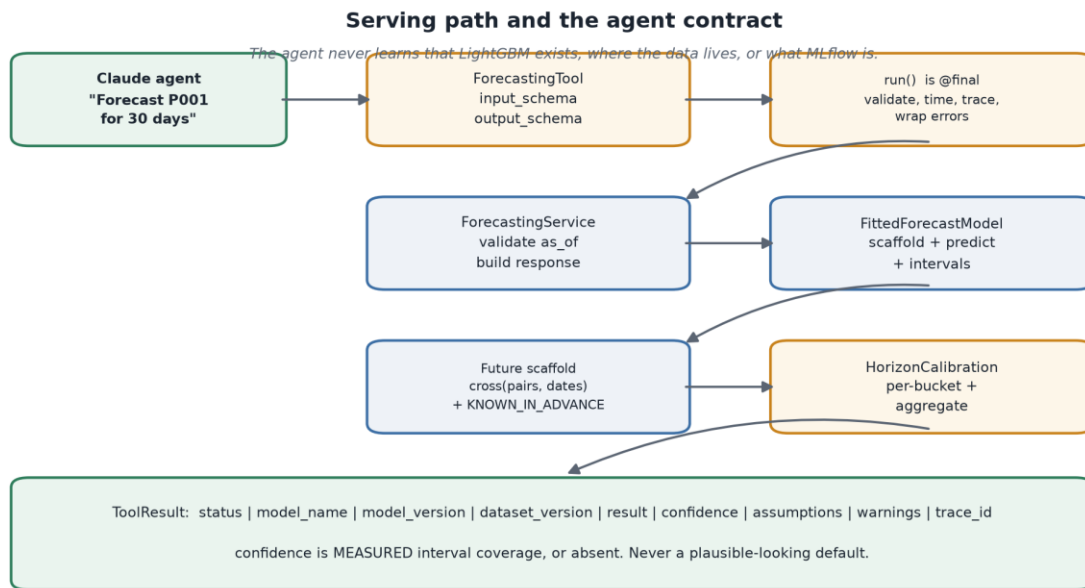


Figure 11 - What the agent sees, and what it is deliberately never told.

The agent should be able to state a number with its uncertainty and re-plan around a refusal. It should **not** need to know that LightGBM exists, where the parquet lives, how features are built, or what MLflow is - and none of that appears in the tool schema.

**On confidence.** confidence is measured interval coverage, or it is absent. There is no third option where a plausible-looking number appears because the field exists. A fabricated 0.89 costs nothing to emit and means nothing.

## Part V - Operations

### Setup

```
git clone <repository>
cd agentic-revenue-intelligence

.\tasks.ps1 setup          # uv sync --all-extras, copies .env.example to .env
uv run ari config          # show resolved settings
uv run ari health          # probe every dependency
```

### Generating data

```
uv run ari generate-data --profile dev --seed 42 --validate

# Profiles:
# smoke - 40 products x 30 stores, used by CI and the test suite
# dev   - 300 x 200, the working dataset
# stress - larger, for performance work
```

### Training the models

```
# Step 4 - baseline sales
uv run python scripts/train_baseline.py --profile dev --seed 42
uv run python scripts/train_baseline.py --sample-pairs 200 --no-backtest --no-track

# Step 5 - demand forecasting
uv run python scripts/train_forecast.py --smoke --no-track    # <60s, correctness
uv run python scripts/train_forecast.py --seed 42             # ~8 min, the deliverable
uv run python scripts/train_forecast.py --full                # hours, declared
```

**Sampled runs write elsewhere.** A sampled run persists to `models/*_sampled/`, never to the directory the service loads from. Step 4 learned this the expensive way: a 400-pair verification run silently replaced a model trained on the full panel, and the only visible trace was a smaller calibration count in the metadata sidecar.

### Quality gates

```
.\tasks.ps1 check      # ruff + mypy + pytest
.\tasks.ps1 lint
.\tasks.ps1 typecheck
.\tasks.ps1 test
.\tasks.ps1 security   # bandit
```

### Configuration reference

| File                             | Controls                                                   |
|----------------------------------|------------------------------------------------------------|
| <code>.env</code>                | Environment selection, credentials, paths. Never committed |
| <code>configs/data/*.yaml</code> | Dataset profiles - scale, causal parameters, confounders   |



|                                 |                                                    |
|---------------------------------|----------------------------------------------------|
| configs/features/features.yaml  | Which feature groups each model dataset uses       |
| configs/models/forecasting.yaml | Horizons, sampling, split, embargo, buckets        |
| pyproject.toml                  | Dependencies, ruff, mypy, pytest, bandit, coverage |

Key environment variables:

```
APP__ENVIRONMENT=local | databricks      # selects every seam implementation
DATA__PARQUET_ROOT=data/local/gold
ML__TRACKING_URI=sqlite:///data/local/mlflow.db
LLM__API_KEY=<secret>                    # never hard-coded, never logged
```

Databricks migration

Design only - nothing is implemented, per the project's staging rule. The full design lives in docs/databricks\_migration.md and docs/forecasting\_databricks\_migration.md.

| Moves unchanged                                                                     | Genuinely changes                                                                                                                                                                   |
|-------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Config, splits, conformal, evaluation, estimators, schemas, the whole tool contract | Panel construction (pandas to PySpark), the horizon self-join (to a Delta join), the future scaffold, model loading (joblib to Unity Catalog), monitoring (to Lakehouse Monitoring) |

**What migration would not fix.** The stockout-exclusion bias is a modelling decision, not a scale problem. The frozen competitor price is a data-availability fact. Distributing the statistical models faster would not make them appropriate at product-store grain. These travel unchanged and are named so the migration is not oversold.

## Part VI - Retrospective

### The defect catalogue

Every defect below was found by a test, a measurement, or a diagnostic that existed to be sceptical - not by code review. They are recorded because the pattern is more instructive than any individual bug.

#### Defects that produced wrong numbers silently

| Defect                                                                                                       | How it surfaced                                                       |
|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| Inventory replenishment ordered against on-hand rather than inventory position, so stockouts were impossible | A dataset invariant test on days-of-cover                             |
| A query returned exactly 100,000 of 6.7M rows with no warning                                                | A deliberate check that row counts matched expectations               |
| revenue reached the feature matrix, recovering the target exactly                                            | A blanket correlation check against the target                        |
| Supply features taught the baseline that low stock predicts low demand                                       | The stockout recovery diagnostic against latent demand                |
| sample_pairs=N loaded ~6.9x N series via an accidental cross product                                         | Measuring the loaded series count against the request                 |
| Sampling was not reproducible - DuckDB returns GROUP BY rows unordered                                       | A determinism test comparing two identical calls                      |
| Categorical dtypes inferred per frame gave inconsistent integer codes                                        | XGBoost refused an unseen category; LightGBM had accepted it silently |
| XGBoost mis-regularised by 38x due to Hessian-scaled min_child_weight                                        | A bias of +58% that could not be explained by overfitting             |

#### Defects in the diagnostics themselves

Three cases where the measuring instrument was wrong rather than the model. These are the most dangerous kind, because they produce confident, incorrect verdicts.

| Defect                                                                                                                                                      | Consequence                                                                                                                                                                              |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| The stockout criterion compared the baseline to <b>latent</b> demand, ignoring that stockouts are endogenous and latent demand during one runs 1.57x normal | All six candidates were disqualified. The criterion, not the models, was wrong                                                                                                           |
| explain.py grouped the 364-day seasonal anchor with lag_1 under one 'demand history' family                                                                 | The combined share rose with horizon and looked exactly like the leakage alarm the function documents. Two features with opposite horizon profiles had been averaged into one diagnostic |
| The scalability claim said per-series ETS was 'hours per backtest fold'                                                                                     | Measurement showed 0.25s per fit - about 25 minutes for a full pass. Expensive, not infeasible. The claim was corrected and the argument moved to where the evidence supports it         |

## Process defects

| Defect                                                                                      | Rule now followed                                                      |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------|
| A three-hour training run lost to an MLflow store rejection raised after training completed | Write the report before tracking; tracking failures never fail the run |
| A sampled verification run silently overwrote a full-panel model                            | Sampled runs write to their own directory                              |
| Regex patching of Python source deleted a method parameter                                  | Never regex source code - use the editor                               |
| New-Item -Force truncated existing __init__.py files                                        | Check existence before creating                                        |
| Set-Content corrupted a UTF-8 notebook                                                      | Use the file-writing tools or Python for content, never Set-Content    |

## Known limitations

| Limitation                                         | Consequence                                                                                                         |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Excluding stockout targets biases the forecast low | Stockouts are endogenous, so this removes part of the high-demand tail. Measured against latent demand and reported |
| No inventory signal in either model                | Both forecast <b>demand</b> . A replenishment planner wants a different model                                       |
| Competitor prices frozen at the forecast origin    | A competitor move inside the horizon is invisible                                                                   |
| No cannibalisation or halo modelling               | A promotion on one SKU distorts its substitutes' numbers, unattributed                                              |
| Promotional baseline cannot be point-validated     | The store-level promo responsiveness multiplier is latent and unpublished, so validation there is directional only  |
| Cannot forecast past 2025-12-31                    | A data limitation. Refused explicitly rather than faked                                                             |
| Conformal assumes exchangeability                  | A trend violates it mildly; coverage is measured per bucket so a shortfall surfaces                                 |
| Trained on synthetic data                          | Real retail has structure this simulator does not reproduce                                                         |

## Roadmap

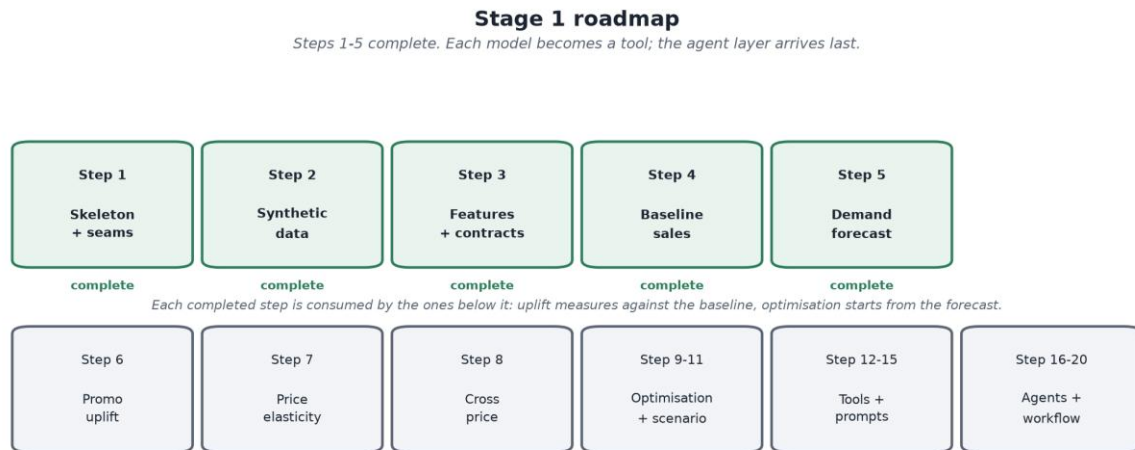


Figure 12 - Each model becomes a tool; the agent layer arrives last.

| Steps | Capability                                                        | Depends on      |
|-------|-------------------------------------------------------------------|-----------------|
| 6     | Promotion uplift - causal, not predictive                         | Step 4 baseline |
| 7     | Price elasticity - using the cost instrument                      | Steps 2-3       |
| 8     | Cross-price elasticity and cannibalisation                        | Step 7          |
| 9-11  | Price optimisation, trade promo optimisation, scenario simulation | Steps 5-8       |
| 12-15 | Tool layer, prompts, memory, guardrails                           | All models      |
| 16-20 | Supervisor, critic, LangGraph workflow, evaluation                | Step 13+        |

## Talking points

The parts of this project worth discussing are rarely the model scores. They are the decisions where a plausible alternative was rejected for a stated reason.

- **Why a structural causal model rather than sampled data.** Because it makes every later claim checkable. Almost no real project can validate a demand model against true demand; this one can, and that is what caught the supply-feature defect.
- **Why availability classes rather than a global as-of clamp.** Because the naive rule is wrong in a way that looks safe. Clamping the promotion calendar would have made forecasting impossible while appearing more rigorous.
- **Why the noise floor is reported alongside every accuracy figure.** Because a bare WMAPE is uninterpretable. 40% reads as poor until you know 35% is unlearnable; then it reads as 1.15x optimal.
- **Why the leakage suite plants a bug.** Because a test that has never failed is indistinguishable from a test that does nothing.
- **Why the forecaster refuses to forecast past the calendar.** Because the alternative produces a number that is systematically low and looks exactly like a real forecast.
- **Why confidence is measured coverage or absent.** Because a fabricated confidence score is the easiest thing in the system to emit and the hardest for a downstream consumer to detect.

- **Why XGBoost's first result was thrown out.** Because a 38x regularisation difference is a configuration error, and reporting it as a model comparison would have been a confident falsehood.

**The through-line.** Every mechanism in this platform exists because a specific plausible mistake would otherwise be invisible. That is the argument for the whole design: not that the models are accurate, but that when they are wrong, something says so.